

Machines à états finis

Table des matières

Définition.....	2
Exemples.....	2
Le réveil matin.....	2
Machine espresso.....	3
Implémentation de la machine espresso.....	5
Références.....	8

Document écrit par Stéphane Gill

Mai 2022



Ce document est soumis à la licence Creative Commons Attribution 3.0 Canada. Permission vous est donnée de copier, modifier et redistribuer des copies de ce document, dans les conditions fixées par la licence, tant que cette note apparaît clairement.

Le célèbre mathématicien et informaticien Alan Turing publia une étude portant sur les machines à états finis en 1936, bien avant que le premier ordinateur digital programmable n'existe. La machine de Turing, baptisée ainsi en l'honneur de Turing, passe successivement par différents états de manière programmée sur la base de données lues sur une bande et écrites sur cette même bande. Cette notion fondamentale qui touche au fonctionnement de l'ordinateur est encore valable actuellement puisque tous les processeurs présents dans nos ordinateurs sont en fait des machines de Turing qui passent d'un état à l'autre à la cadence de l'horloge. Cependant, les machines à états qui peuvent être modélisées par un graphe de transition se prêtent mieux aux applications pratiques. Du fait qu'elles ne disposent que d'un nombre fini d'états, on les appelle des machines à états finis (ou automates à états finis).

Définition

Une machine à états finis est un système dynamique fonctionnant par cycles, dans lequel chaque transition entre l'état courant et un autre état est déclenchée par un événement externe.

Note :

- La machine ne peut se trouver que dans un seul état à la fois et compte un nombre fini d'états, illustrés dans un **diagramme d'états**.
- L'ordre des transitions est quant à lui défini dans un registre d'états.

Exemples

Chaque jour, nous sommes tous confrontés à des appareils et machines qui peuvent être considérés comme des automates parmi lesquels on trouve les distributeurs automatiques de boissons ou de billets de banque, les machines à laver et tant d'autres. Un ingénieur développant de telles machines se doit de comprendre clairement qu'elles effectuent leur tâche en passant de l'état courant à un état successeur. Ces transitions d'état sont déterminées par les entrées de l'automate, à savoir les données en provenance des capteurs ou des interrupteurs formant l'interface de commande. En fonction des données en entrée, l'opérateur actionne certains acteurs lors de chaque transition d'états tels que des moteurs, des pompes, des témoins lumineux qui constituent les sorties de l'automate.

Le réveil matin

Considérons un réveil-matin simplifié :

- on peut mettre l'alarme « on » ou « off » ;

- quand l'heure courante devient égale à l'heure d'alarme, le réveil sonne sans s'arrêter ;
- on peut interrompre la sonnerie.

Le réveil a clairement deux états distincts : Désarmé (alarme « off ») ou Armé (alarme « on »). Une action de l'utilisateur permet de passer d'un état à l'autre. On suppose que le réveil est bien désarmé au départ. Le fait de sonner constitue un nouvel état pour le réveil. Il s'agit bien d'une période de temps durant laquelle le réveil effectue une certaine activité (sonner) qui dure jusqu'à ce qu'un événement vienne l'interrompre. Le passage de l'état Armé à l'état Sonnerie est déclenché par une transition due à un changement interne. En revanche, le retour de l'état Sonnerie à l'état Armé ne s'effectue que sur un événement utilisateur.

Bien qu'il soit possible de décrire le fonctionnement d'un réveille-matin en prose, un diagramme d'état est bien plus clair et explicite. Un tel graphe est formé de cercles représentant les différents états (nœuds du graphe) et de flèches (arête orientée) passant d'un état à l'autre et formalisant les transitions d'états. Les flèches sont étiquetées par les entrées / sorties qui causent la transition en question. Il est également important de définir l'état initial de la machine lorsqu'elle est mise en fonction. Du fait que n'importe quelle touche peut être actionnée dans n'importe lequel des états, toutes les entrées doivent être envisagées à chaque état. Si aucune action n'est effectuée, la sortie est omise.

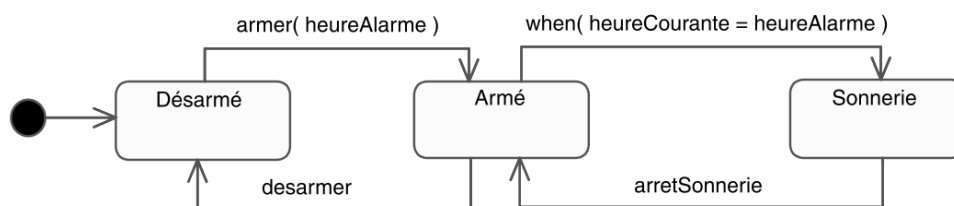


Diagramme d'états du réveille-matin

Machine espresso

Considérons maintenant une machine à café espresso peut se trouver dans trois états différents :

- elle peut être éteinte (OFF),
- allumée et en attente (STANDBY)
- ou en train de pomper et chauffer de l'eau pour préparer un espresso (WORKING).

L'interface de contrôle de la machine comporte quatre boutons-poussoirs : turnOn, turnOff, work, et stop.

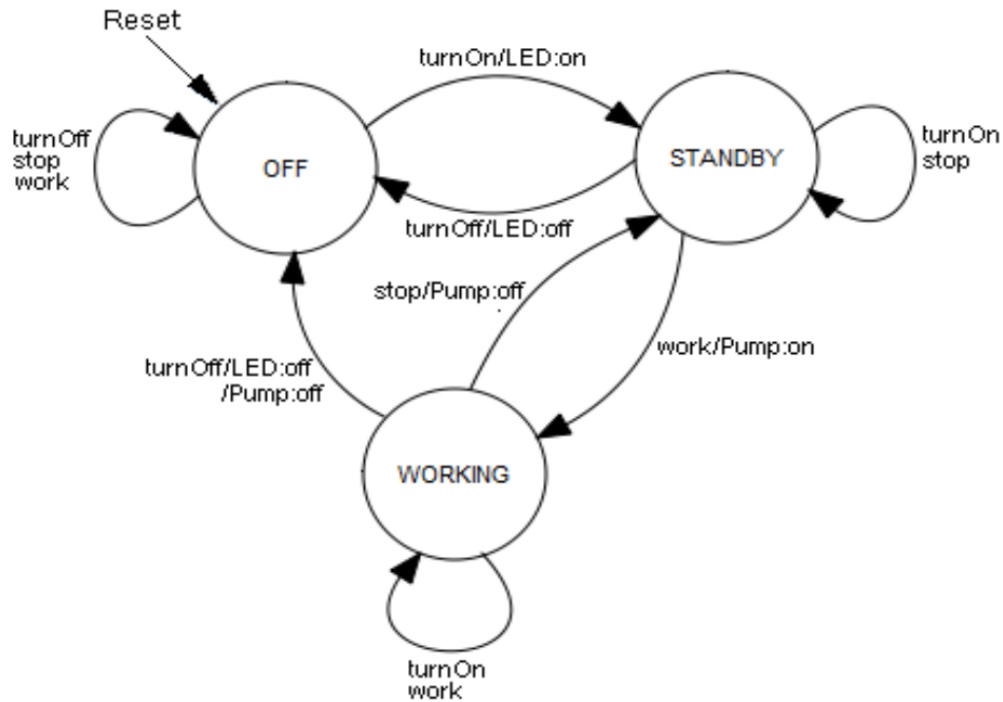


Diagramme d'états de la machine espresso

On résume souvent le comportement de la machine dans un tableau qui spécifie pour chaque état s et chaque entrée t (ici les boutons actionnés) l'état successeur s' . L'état initial est dénoté par une étoile.

Table de transition:

$t =$ $s =$	OFF(*)	STANDBY	WORKING
turnOff	OFF	OFF	OFF
turnOn	STANDBY	STANDBY	WORKING
stop	OFF	STANDBY	STANDBY
work	OFF	STANDBY	WORKING

En termes mathématiques, on peut dire que l'état successeur s' est une fonction de l'état courant et de l'entrée t : $s' = F(s, t)$. La fonction F est appelée fonction de transition.

On peut également spécifier dans un tel tableau les sorties correspondant à chaque état et chaque donnée en entrée :

Table de sortie:

t = s =	OFF(*)	STANDBY	WORKING
turnOff	-	LED éteinte	LED éteinte, Pompe allumé
turnOn	LED allumée	-	-
stop	-	-	Pump éteinte
work	-	Pump allumée	-

À nouveau, on peut dire que, mathématiquement, la sortie g est une fonction de l'état courant et des entrées : $g = G(s, t)$. G est appelée fonction de sortie.

Tout ceci, à savoir les différents états, les valeurs en entrée, les valeurs en sortie ainsi que les fonctions de transition et de sortie forment une machine de Mealy.

Implémentation de la machine espresso

Exemple 1 : Implémentation de la machine espresso avec des chaînes de caractères

```

1  def getEvent():
2      print("Choisir: 1.Stop, 2.Work, 3.turnOff ou 4.turnOn")
3      choix = int(input("Choix ? "))
4      if choix == 1:
5          return "stop"
6      if choix == 2:
7          return "work"
8      if choix == 3:
9          return "turnOff"
10     if choix == 4:
11         return "turnOn"
12     return ""
13
14  etat = "OFF"
15  while True:
16      print("\nEtat: " + etat)
17      event = getEvent()
18      if event == "turnOff":
19          if etat == "STANDBY":
20              etat = "OFF"
21              print("DEL off")
22          if etat == "WORKING":
23              etat = "OFF"
24              print("DEL et pompe off")
25      elif event == "turnOn":
26          if etat == "OFF":
27              etat = "STANDBY"
28              print("DEL on")
29      elif event == "stop":
30          if etat == "WORKING":

```

```

31         etat = "STANDBY"
32         print("Pompe off")
33     elif event == "work":
34         if etat == "STANDBY":
35             etat = "WORKING"
36             print("Pompe on")

```

Exemple 2 : Implémentation de la machine espresso avec des énumérations

```

1  from enum import Enum
2
3  class Etat(Enum):
4      OFF = 1
5      STANDBY = 2
6      WORKING = 3
7
8  class Event(Enum):
9      stop = 1
10     work = 2
11     turnOff = 3
12     turnOn = 4
13
14     def getEvent():
15         print("Choisir: 1.Stop, 2.Work, 3.turnOff ou 4.turnOn")
16         choix = int(input("Choix ? "))
17         if choix == 1:
18             return Event.stop
19         if choix == 2:
20             return Event.work
21         if choix == 3:
22             return Event.turnOff
23         if choix == 4:
24             return Event.turnOn
25         return ""
26
27     etat = Etat.OFF
28     while True:
29         print("\nEtat: " + etat.name)
30         event = getEvent()
31         if event == Event.turnOff:
32             if etat == Etat.STANDBY:
33                 etat = Etat.OFF
34                 print("DEL off")
35             if etat == Etat.WORKING:
36                 etat = Etat.OFF
37                 print("DEL et pompe off")
38         elif event == Event.turnOn:
39             if etat == Etat.OFF:
40                 etat = Etat.STANDBY
41                 print("DEL on")
42         elif event == Event.stop:
43             if etat == Etat.WORKING:

```

```

44         etat = Etat.STANDBY
45         print("Pompe off")
46     elif event == Event.work:
47         if etat == Etat.STANDBY:
48             etat = Etat.WORKING
49             print("Pompe on")

```

Exemple 3 : Implémentation de la machine espresso avec une interface Thinker

```

1  from tkinter import *
2  from enum import Enum
3
4  class Etat(Enum):
5      OFF = 1
6      STANDBY = 2
7      WORKING = 3
8
9  class Event(Enum):
10     stop = 1
11     work = 2
12     turnOff = 3
13     turnOn = 4
14
15  def setEtat(event):
16     global etat
17
18     if event == Event.turnOff:
19         if etat == Etat.STANDBY:
20             etat = Etat.OFF
21             lblCmd.configure(text = "DEL off")
22         elif etat == Etat.WORKING:
23             etat = Etat.OFF
24             lblCmd.configure(text = "DEL et pompe off")
25     elif event == Event.turnOn:
26         if etat == Etat.OFF:
27             etat = Etat.STANDBY
28             lblCmd.configure(text = "DEL on")
29     elif event == Event.stop:
30         if etat == Etat.WORKING:
31             etat = Etat.STANDBY
32             lblCmd.configure(text = "Pompe off")
33     elif event == Event.work:
34         if etat == Etat.STANDBY:
35             etat = Etat.WORKING
36             lblCmd.configure(text = "Pompe on")
37
38     lblEtat.configure(text = etat.name)
39
40
41  etat = Etat.OFF
42  fen1 = Tk()
43

```

```
44 lblEtat = Label(fen1, text=etat.name, fg='red')
45 lblEtat.grid(row = 0, column = 0, columnspan = 2)
46
47 lblCmd = Label(fen1, text="DEL off", fg='blue')
48 lblCmd.grid(row = 0, column = 2, columnspan = 2)
49
50 btnStop = Button(fen1, text='Stop',command=lambda: setEtat(Event.stop))
51 btnStop.grid(row = 1, column = 0)
52
53 btnWork = Button(fen1, text='Work',command=lambda: setEtat(Event.work))
54 btnWork.grid(row = 1, column = 1)
55
56 btnOff = Button(fen1, text='TurnOff',command=lambda: setEtat(Event.turnOff))
57 btnOff.grid(row = 1, column = 2)
58
59 btnOn = Button(fen1, text='TurnOn',command=lambda: setEtat(Event.turnOn))
60 btnOn.grid(row = 1, column = 3)
61
62 fen1.mainloop()
```

Références

- Section 10.6 – Automates finis du livre Concepts de Programmation en Python avec la plateforme TigerJython.
- Chapitre 6 - Modélisation dynamique du livre UML par la pratique.
- Vidéo du cours : <https://youtu.be/eWo9Y534tYA>